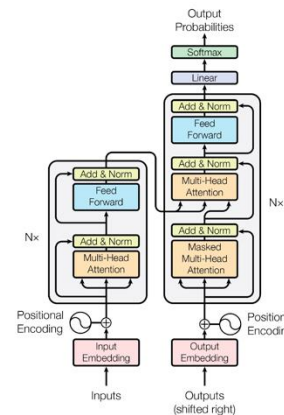


Transformers

COMP6252: Deep Learning Technologies

University of Southampton UK



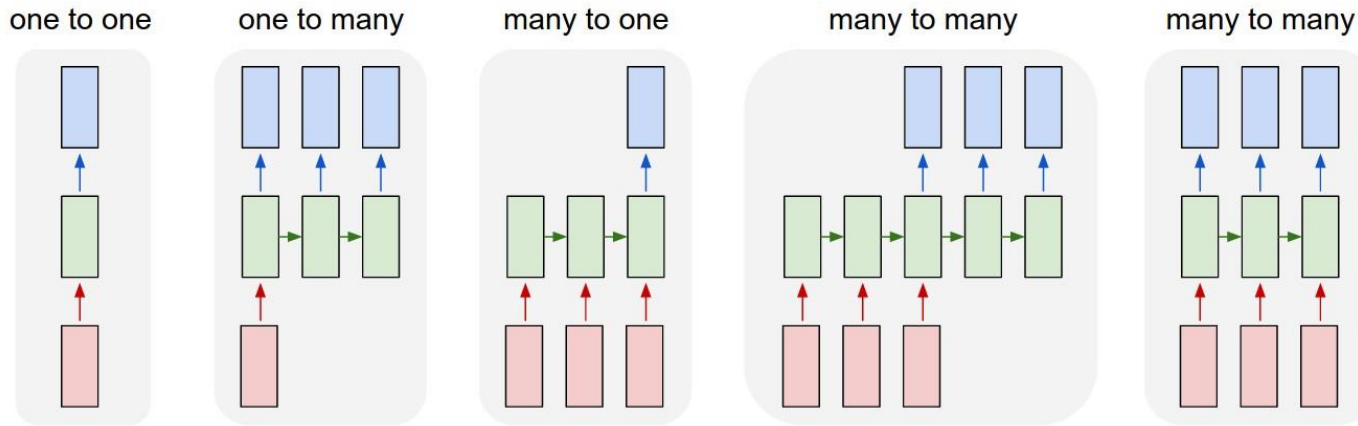
Xiaohao Cai

VLC / ECS / UoS

Content

- Motivation
- Transformers

Why Transformers?



Limitations:

RNN - LSTMs

- Hard to train – time-consuming
- Do not support **parallel processing**
- Performance is not the best anymore in several applications
- ...

Transformers
are able to solve
these problems

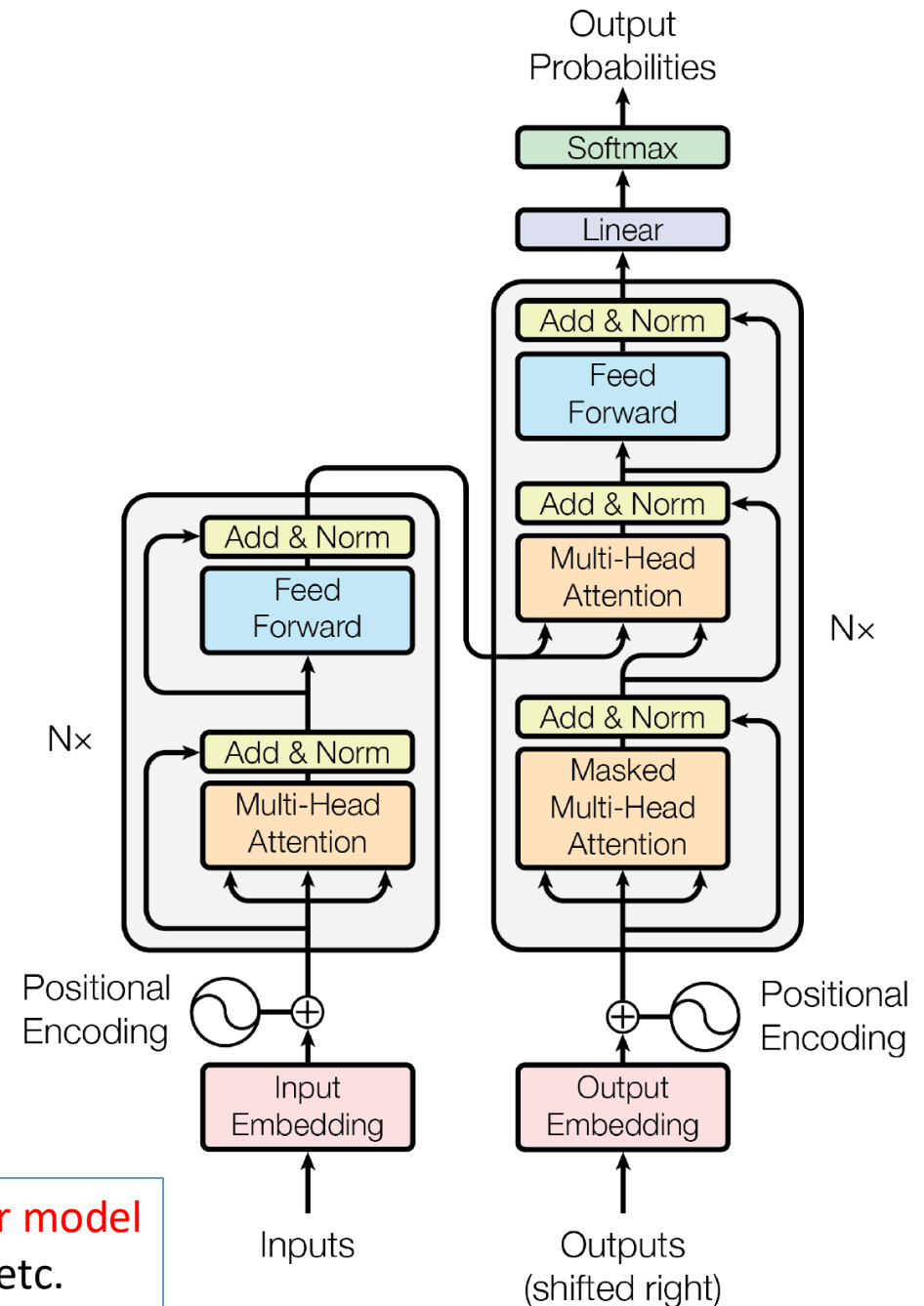
Transformers – Full Architecture

Attention is all you need (2017)

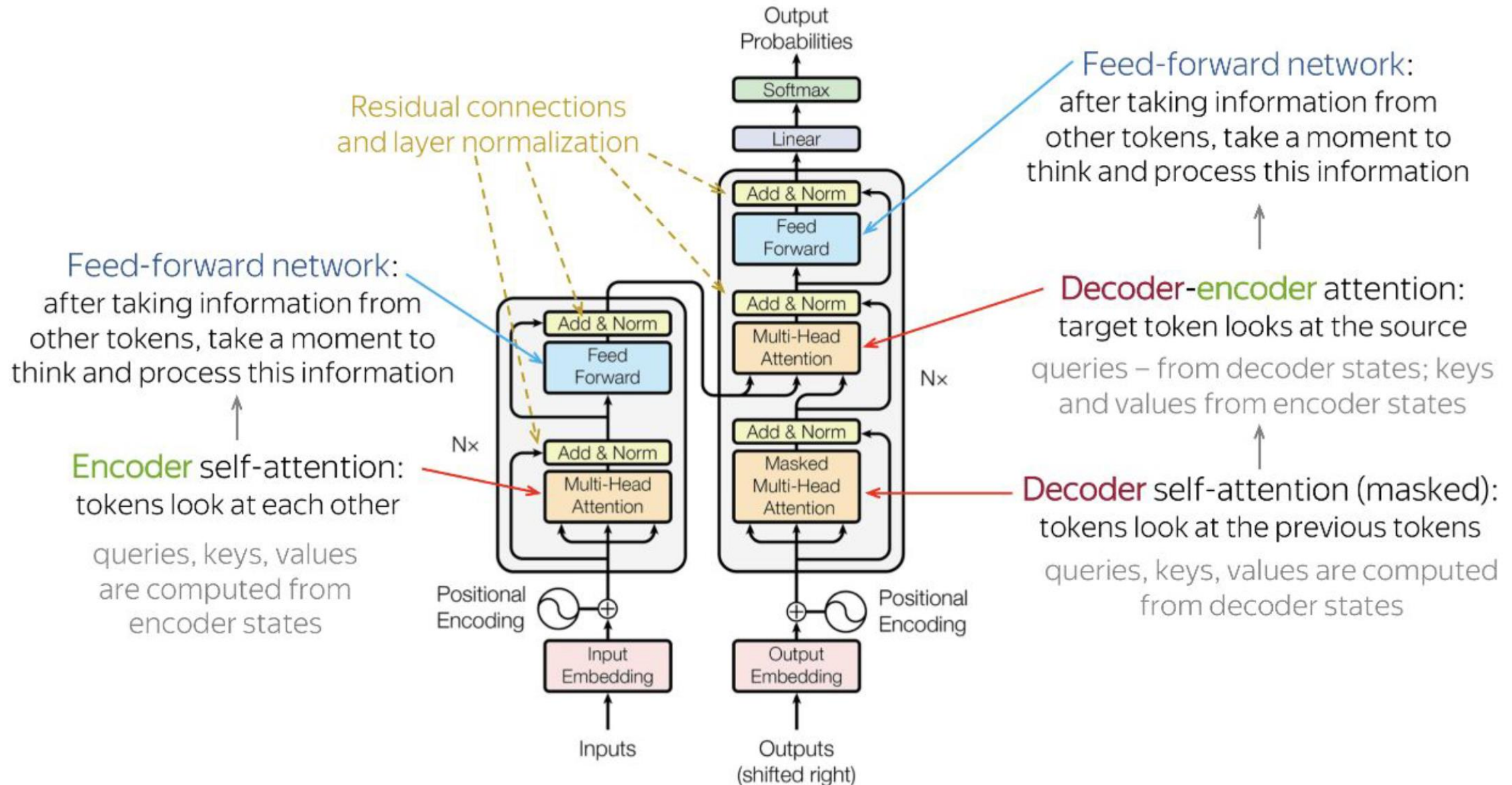
Ah ... too complicated ... !!!



- A new kind of **encoder-decoder model**
- Used for language translation, etc.



Transformers – Full Architecture

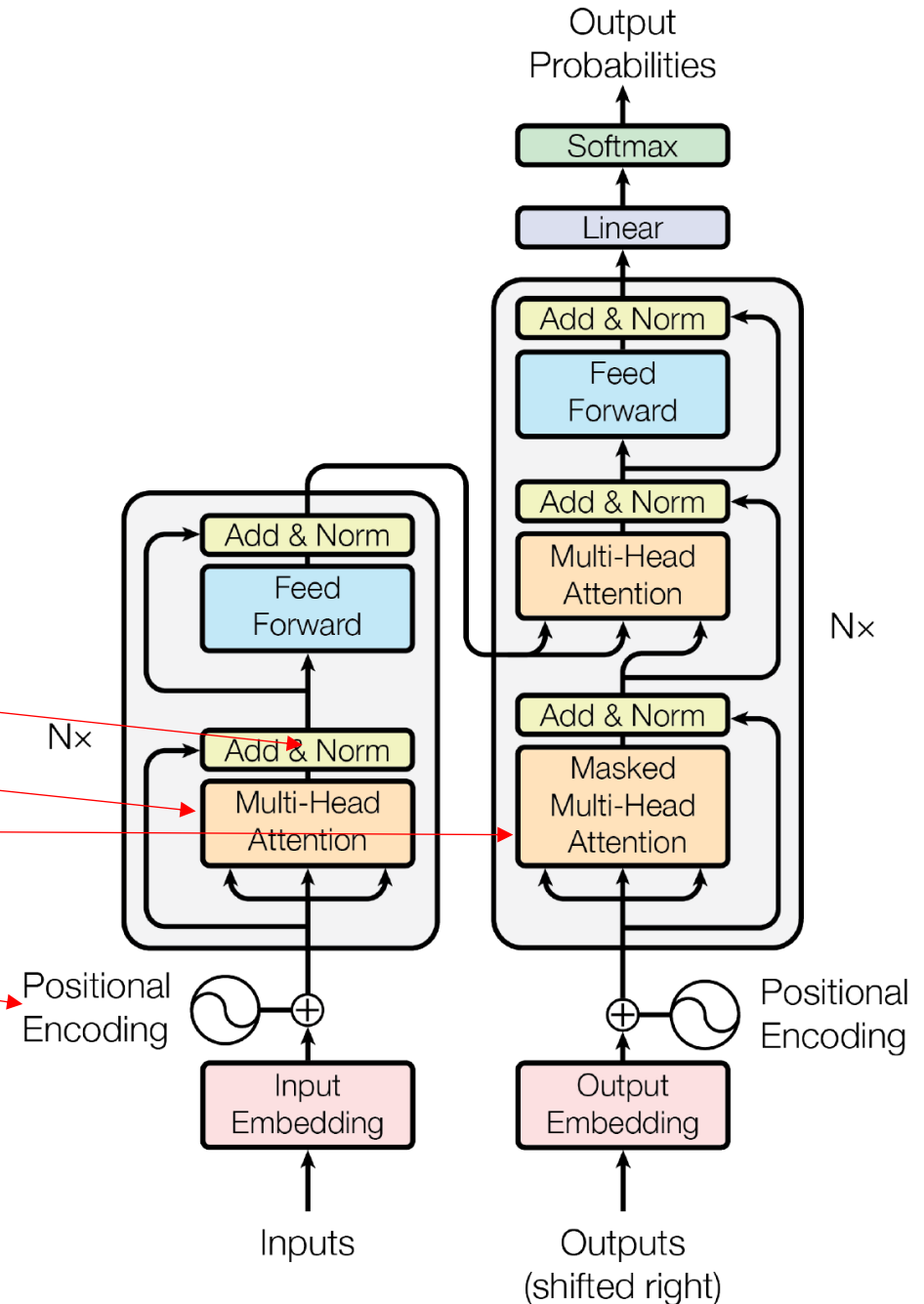


Transformers – Full Architecture

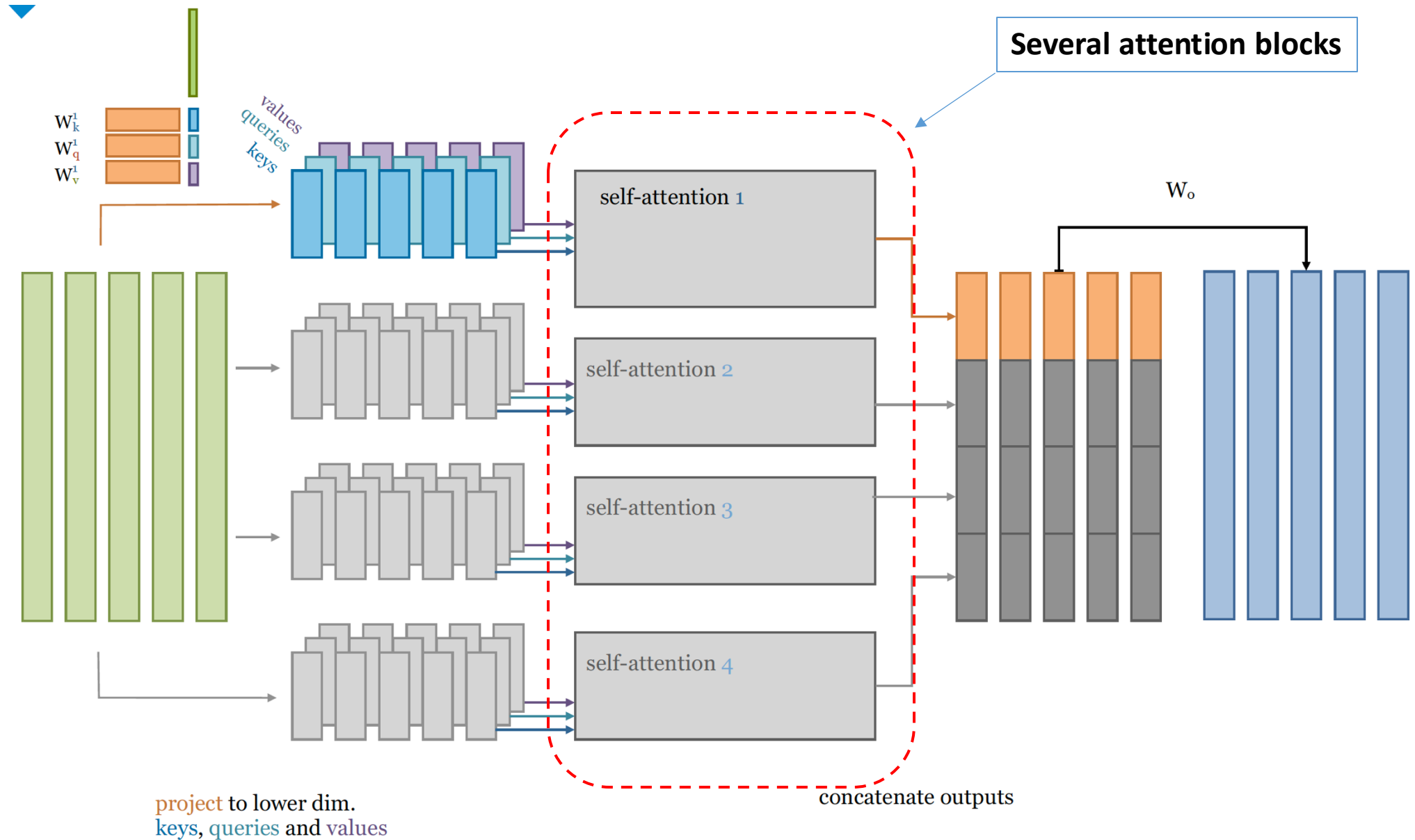
Attention is all you need (2017)

Parts we do not know yet:

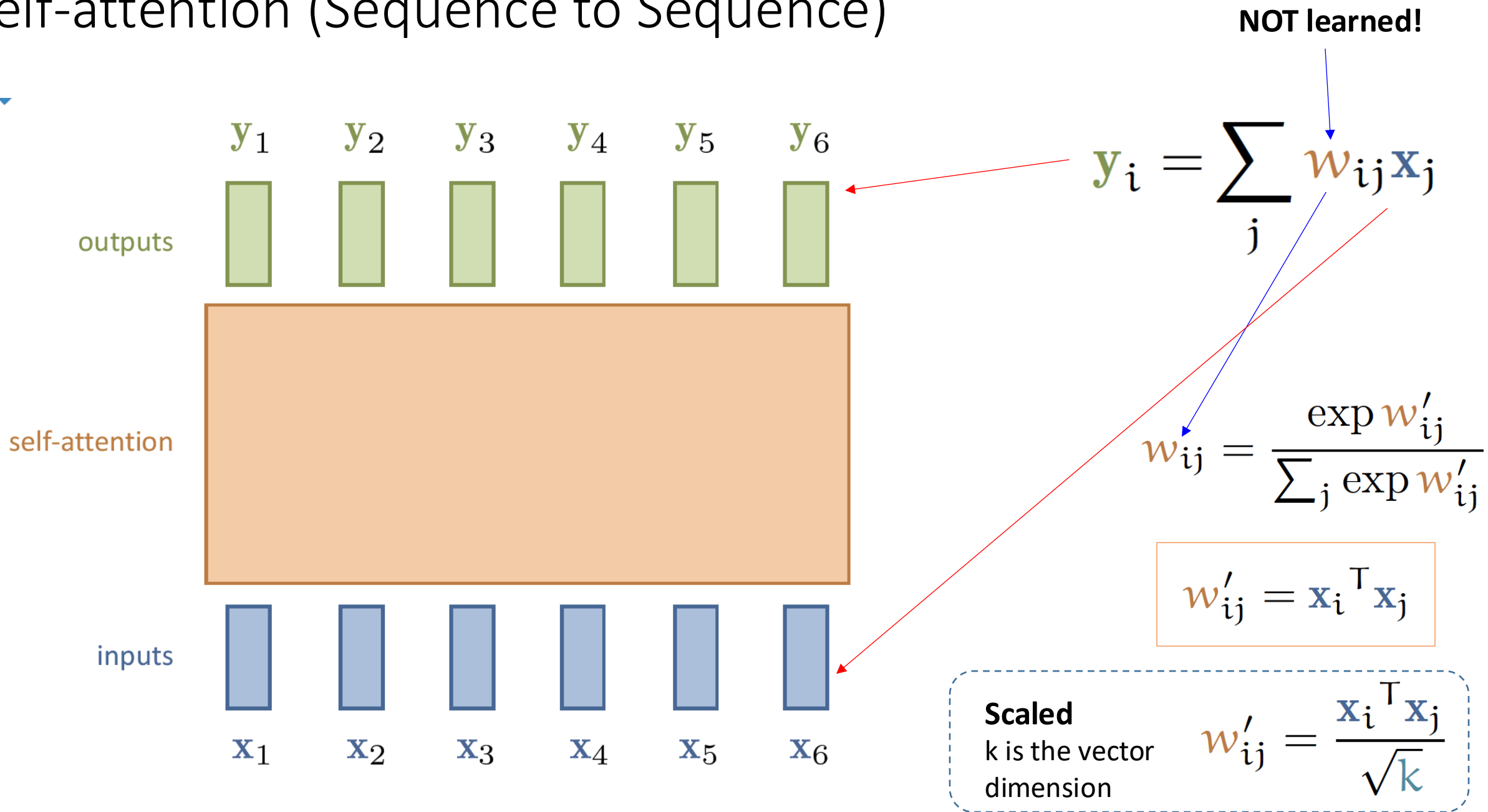
- Layer Normalization
- Multi-Head Attention
- Masked Multi-Head Attention
- Positional Encoding



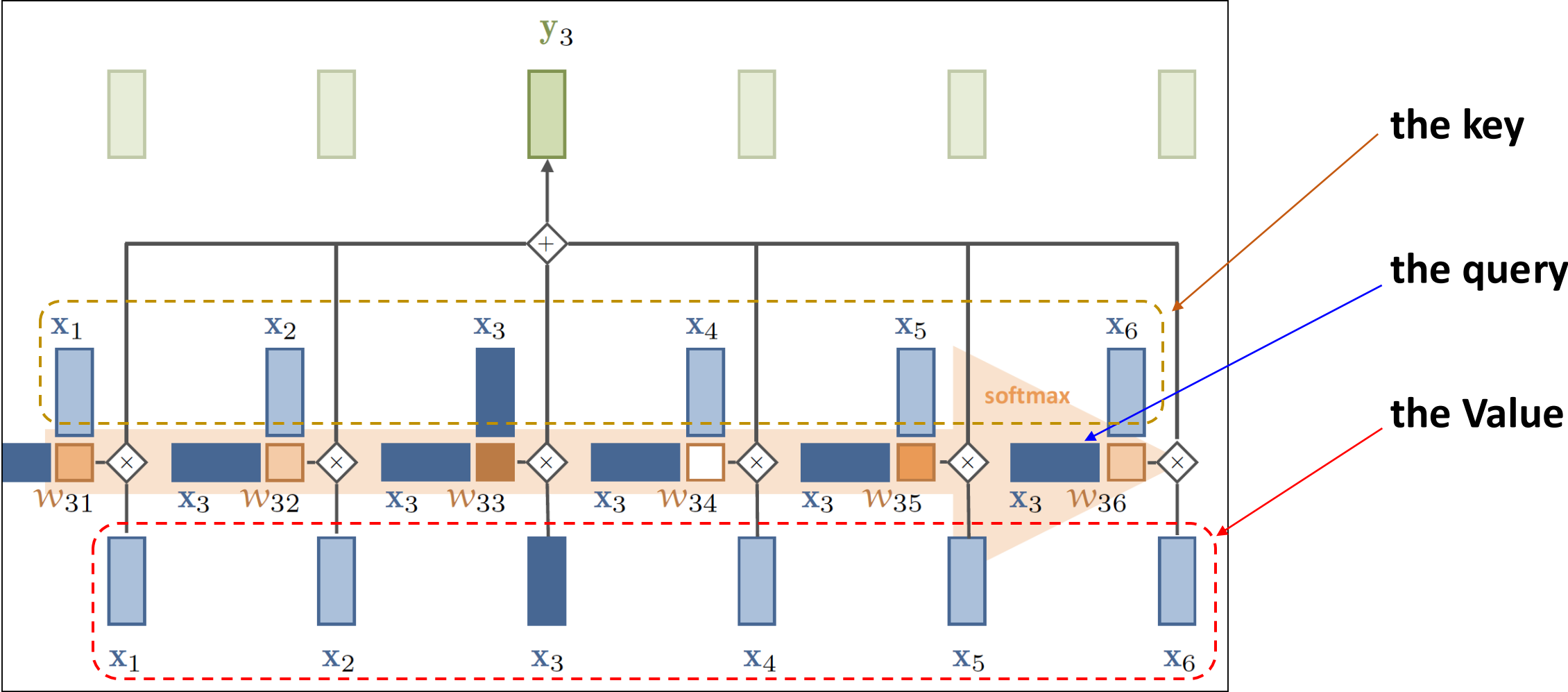
Multi-Head Attention



Self-attention (Sequence to Sequence)

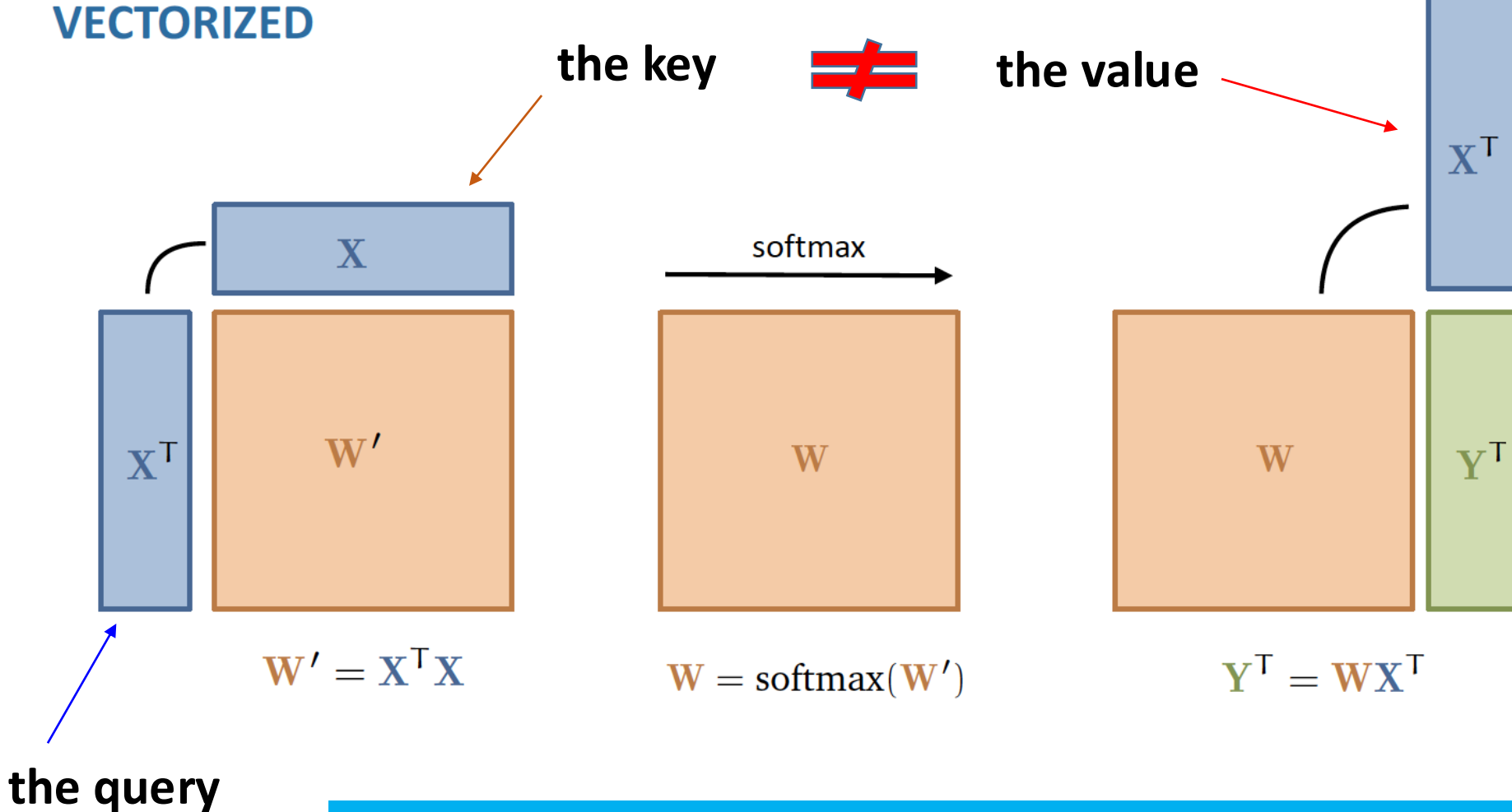


Self-attention



Self-attention

Note that, sometimes,
the key and value
are **different** vectors

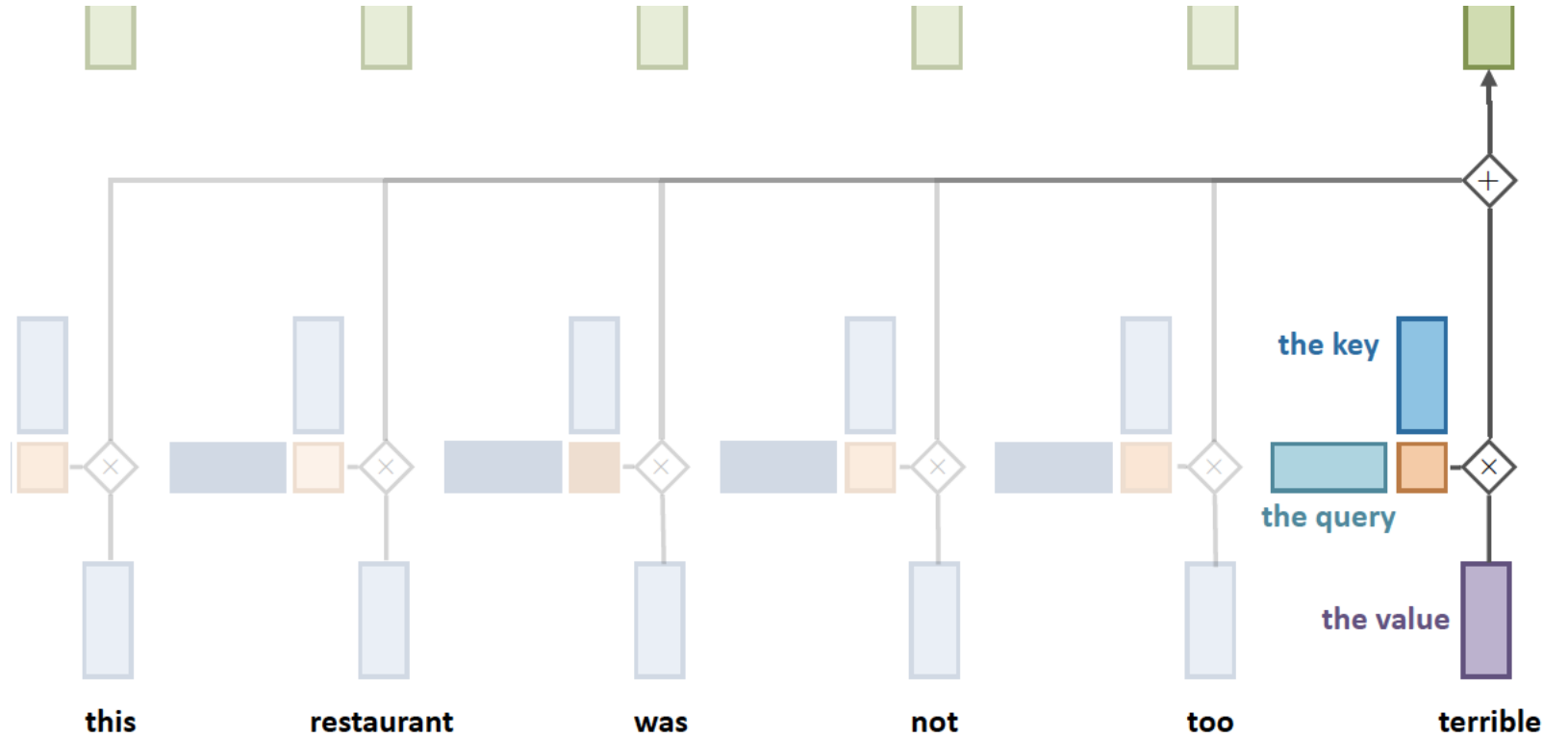


Question: Do we always need the **full** attention weight matrix W ?

Self-attention

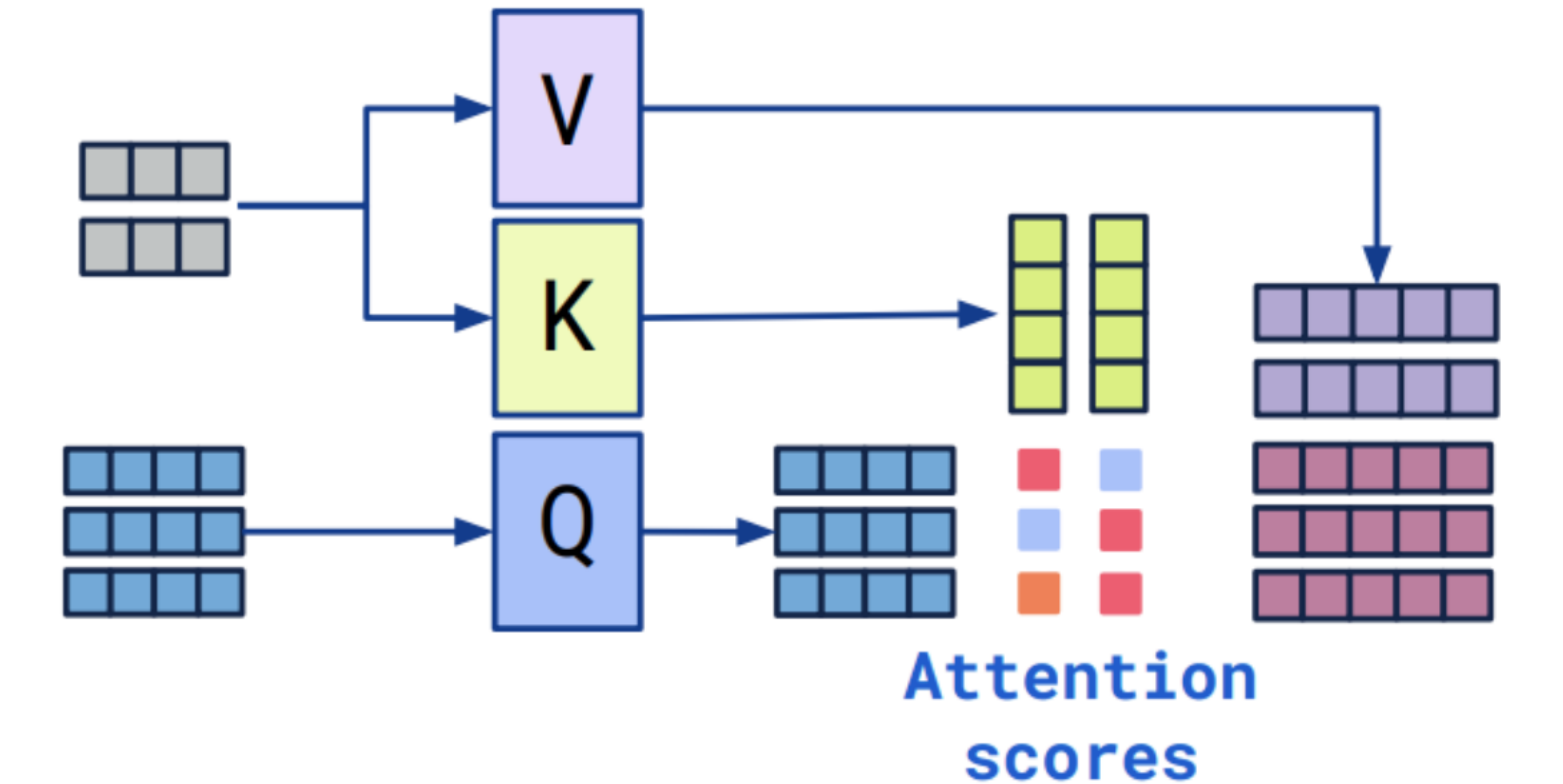


Attention with keys,
queries and values
from the **same set**



Cross-attention

Attention with keys,
queries and values
from **different sets**



Key, Query and Value Transformation

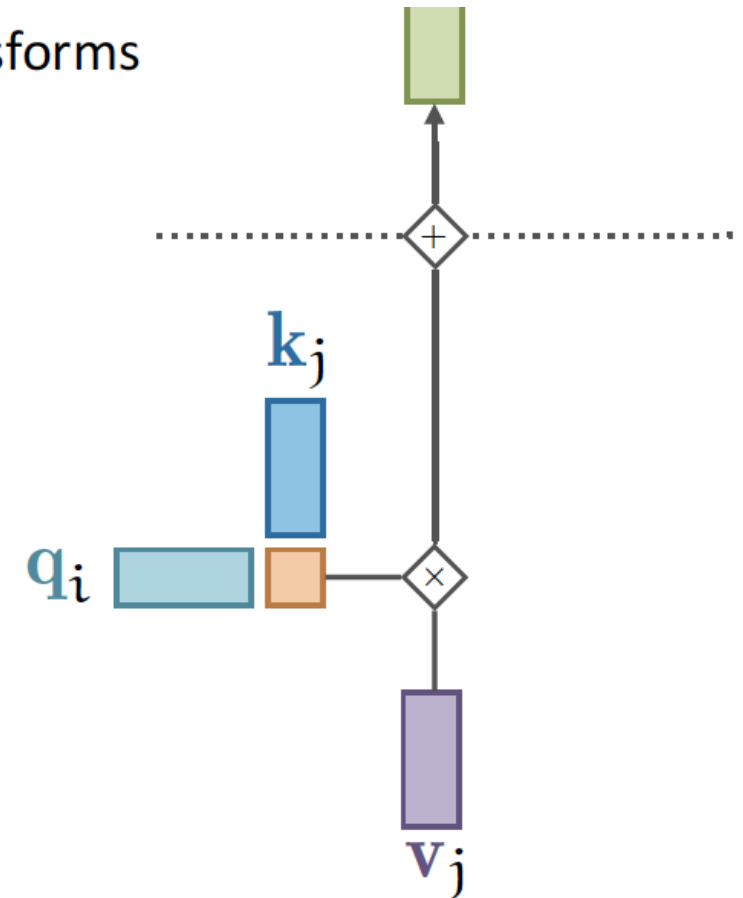
introduce matrices $\mathbf{K}$, $\mathbf{Q}$, $\mathbf{V}$ for linear transforms
and associated biases

$$\mathbf{k}_i = \mathbf{K}\mathbf{x}_i + \mathbf{b}_k$$

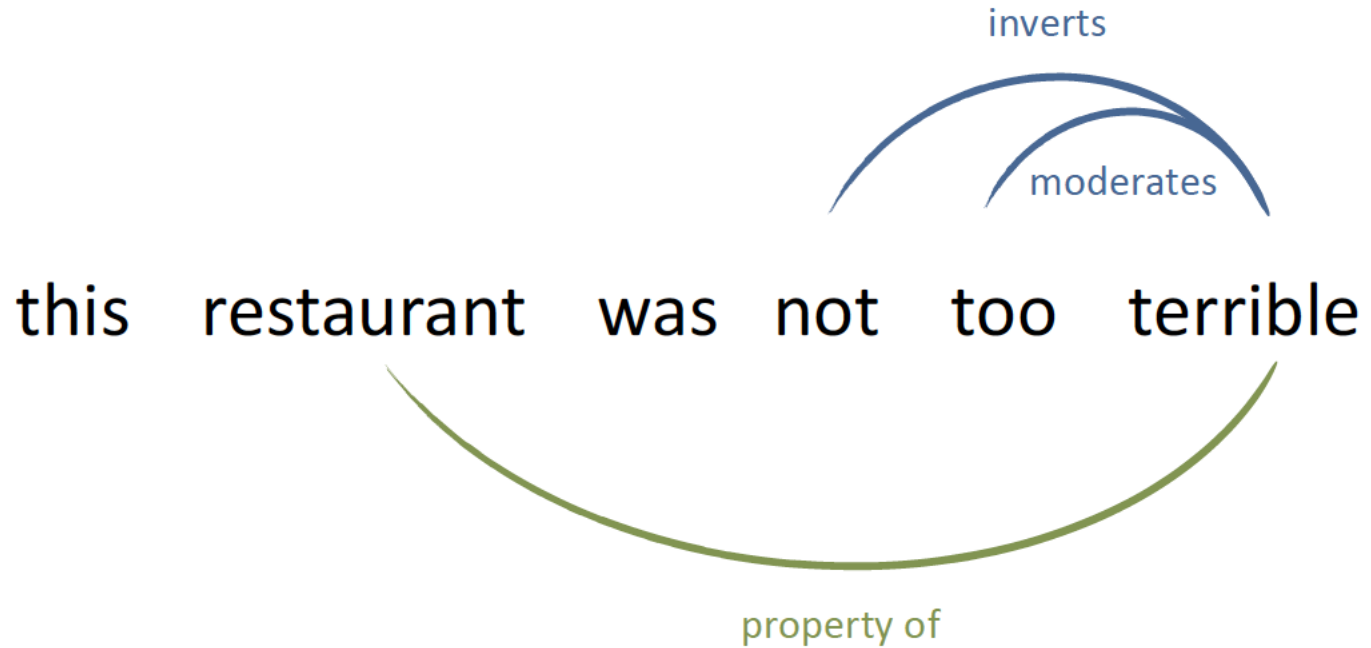
$$\mathbf{q}_i = \mathbf{Q}\mathbf{x}_i + \mathbf{b}_q$$

$$\mathbf{v}_i = \mathbf{V}\mathbf{x}_i + \mathbf{b}_v$$

To give the self attention some more flexibility in determining its behavior, we could multiply each input vector $\mathbf{x}$ by three different parameter matrices, $\mathbf{K}$, $\mathbf{Q}$, and $\mathbf{V}$, which gives us different vectors to act as key, query, and value.

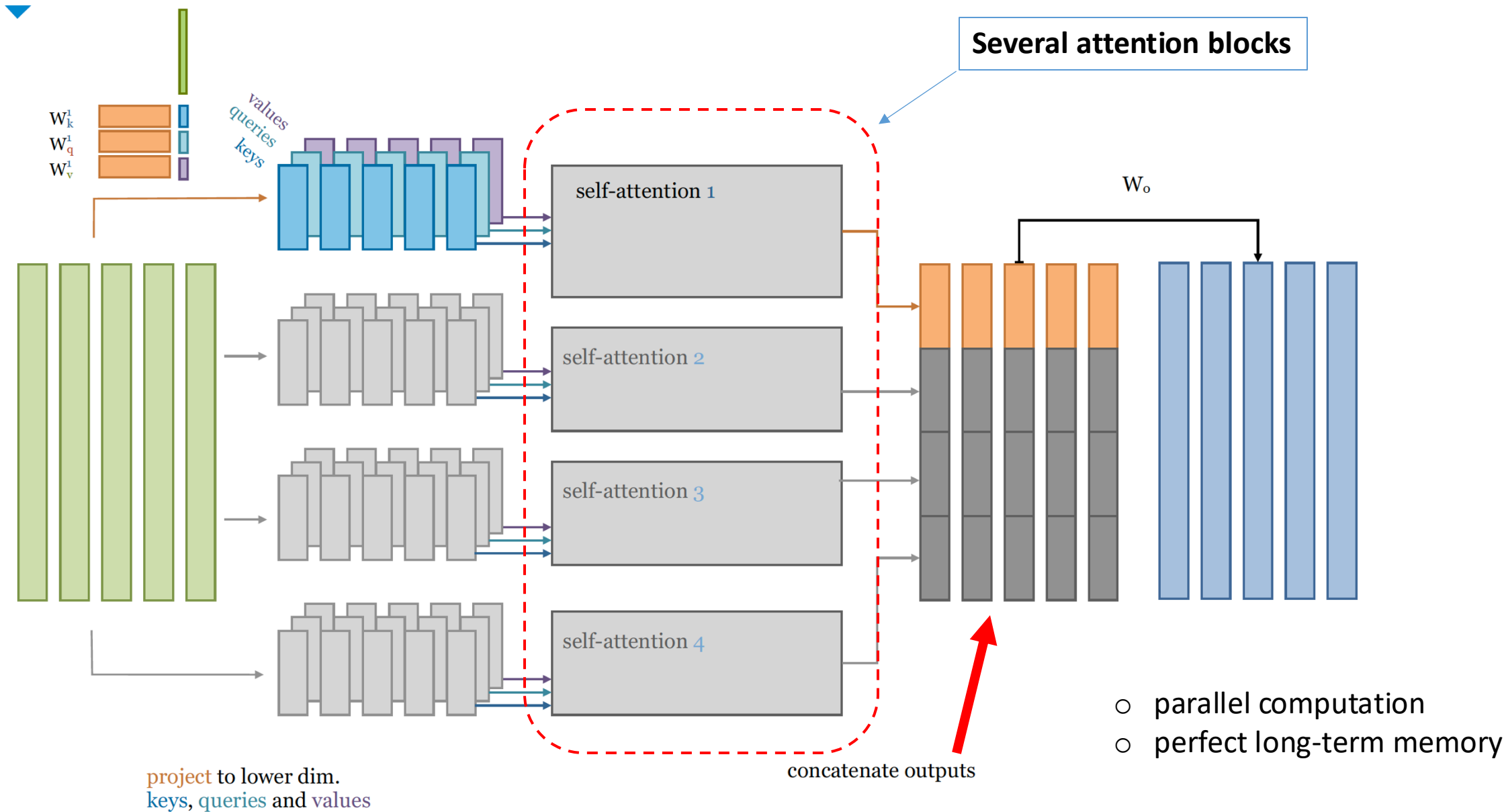


Multi-Head Self-attention



- In many sentences, there are different relations to model.
- The idea behind **multi-head self-attention** is that multiple relations are best captured by different self-attention operations.

Multi-Head Attention

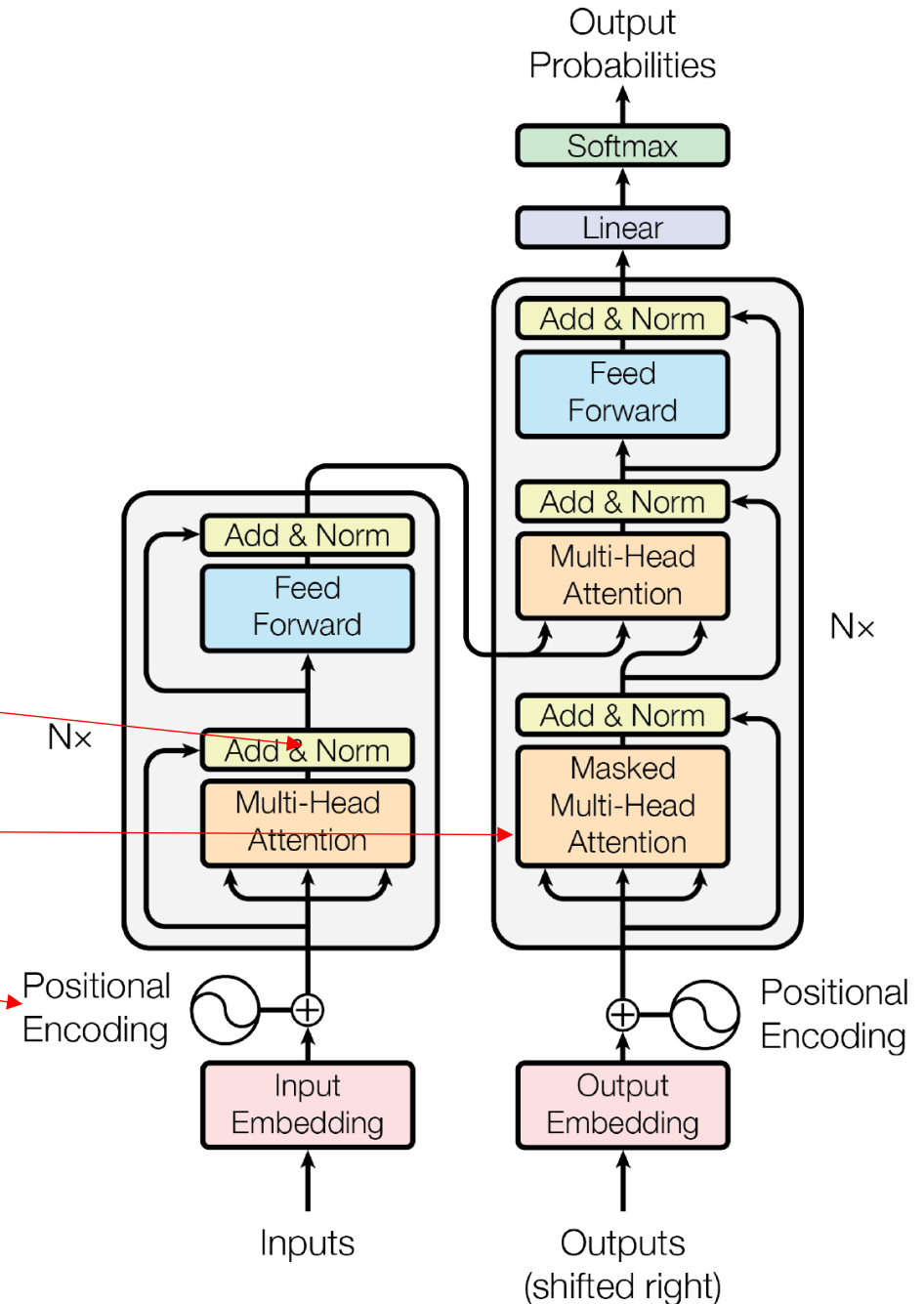


Transformers – Full Architecture

Attention is all you need (2017)

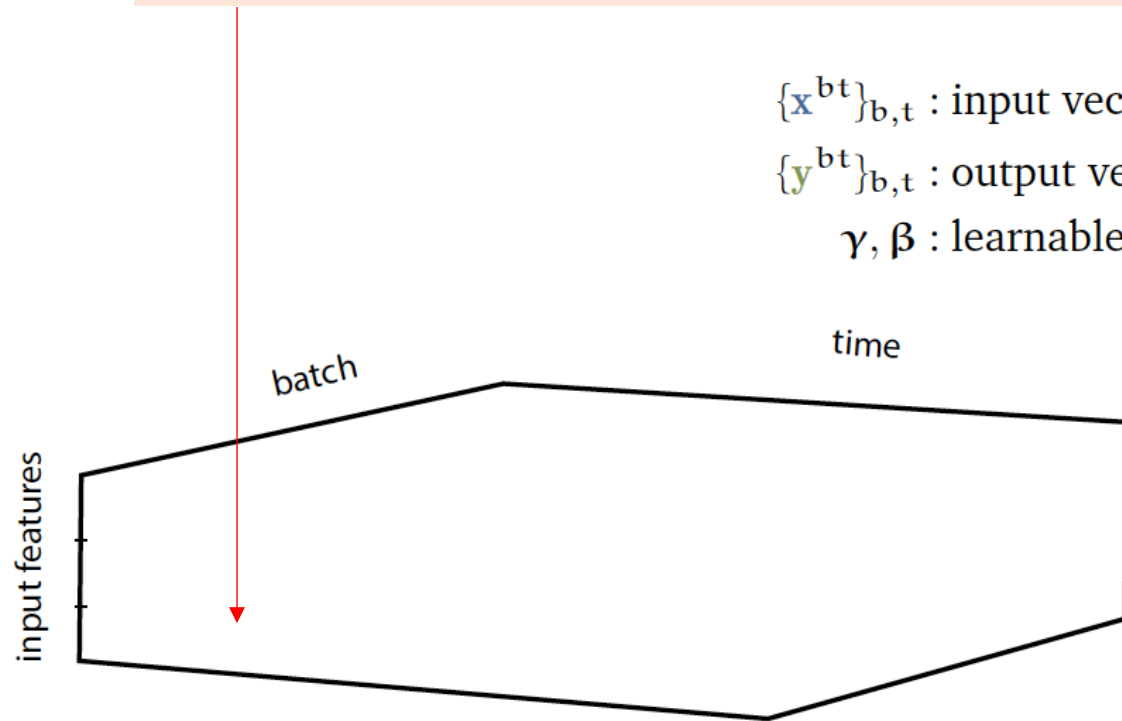
Parts we do not know yet:

- Layer Normalisation
- ~~Multi-Head Attention~~
- Masked Multi-Head Attention
- Positional Encoding



Layer Normalisation

Along this direction – doesn't depend on the batch size!



$\{\mathbf{x}^{bt}\}_{b,t}$: input vectors (one per timestep t and batch instance b) in $\mathbb{R}^d$

$\{\mathbf{y}^{bt}\}_{b,t}$: output vectors (one per timestep t and batch instance b) in $\mathbb{R}^d$

γ, β : learnable parameter vectors

$$\mu^{bt} = \frac{1}{d} \sum_i x_i^{bt} \quad \text{mean over token}$$

$$\sigma^{bt} = \frac{1}{d} \sum (x_i^{bt} - \mu)^2 \quad \text{variance over token}$$

$$\hat{\mathbf{x}}^{bt} = \frac{\mathbf{x}^{bt} - \mu^{bt}}{\sqrt{\sigma^{bt} + \epsilon}} \quad \text{standardize}$$

$$\mathbf{y}^{bt} = \gamma^T \hat{\mathbf{x}}^{bt} + \beta \quad \text{rescale}$$

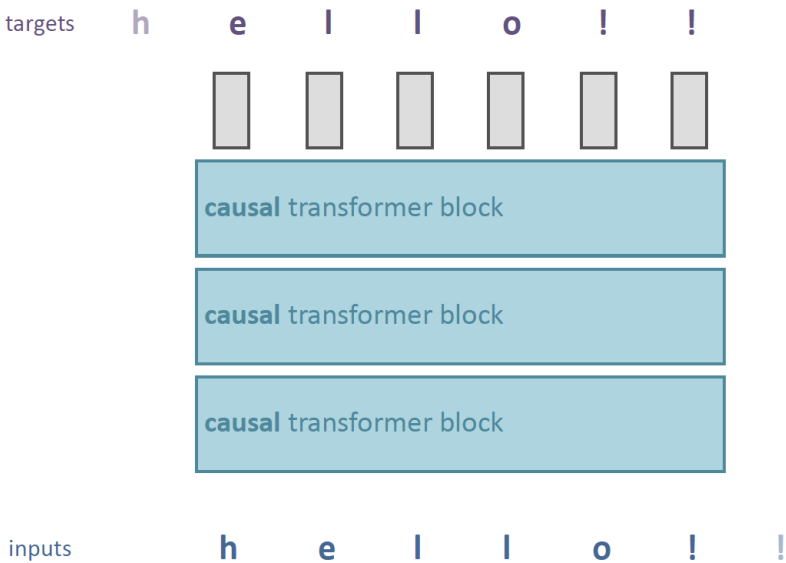
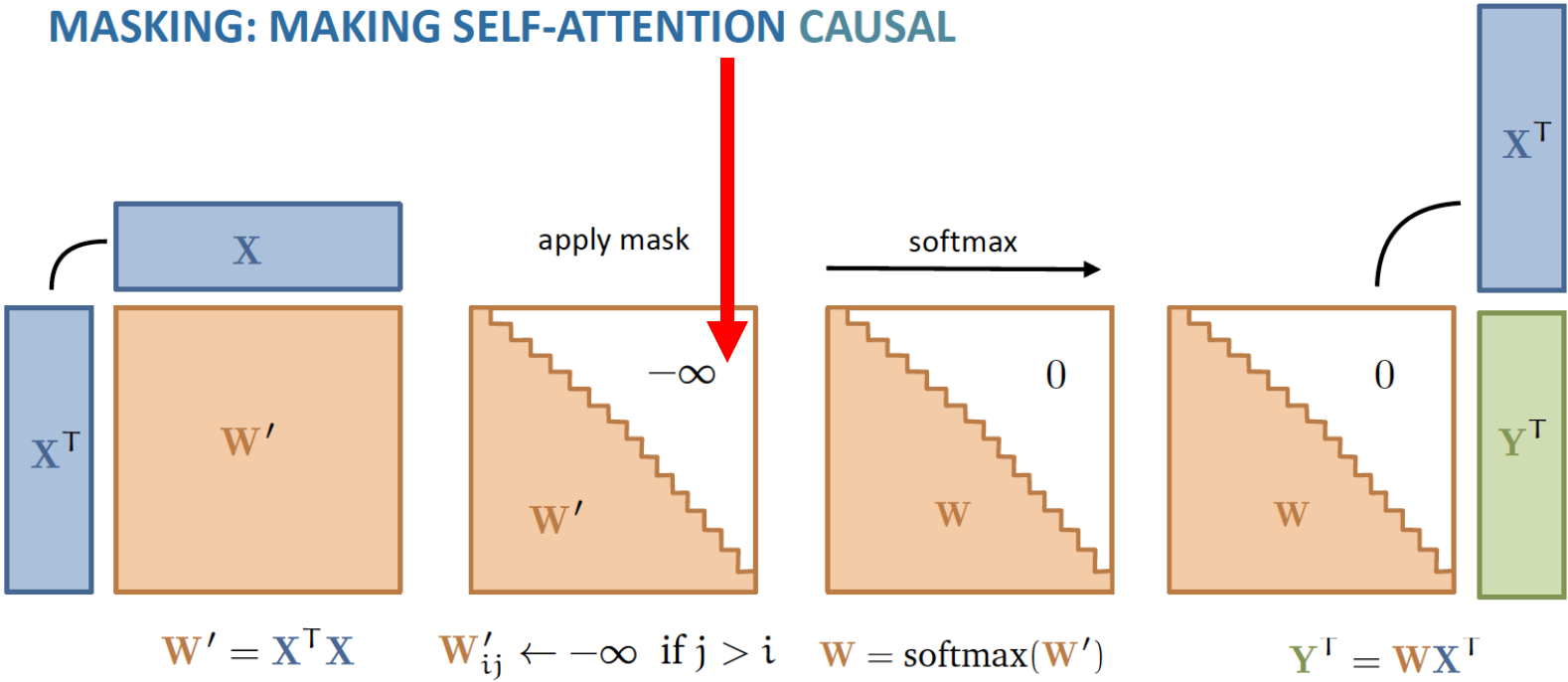


normalize to $N(0, 1)$

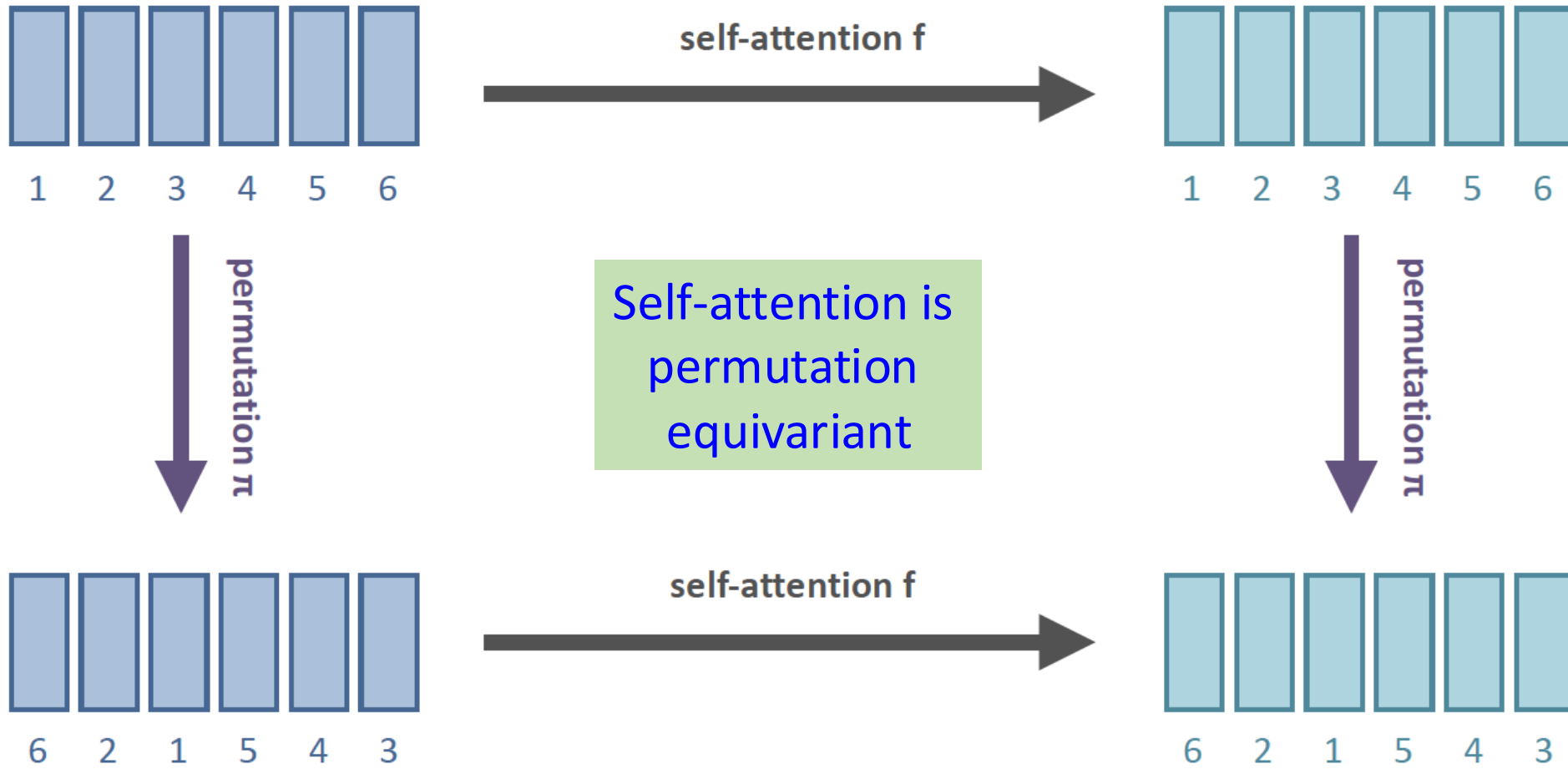
Masked Multi-Head Attention

This makes the whole block **causal**, and be able to **predict the future from the past**.

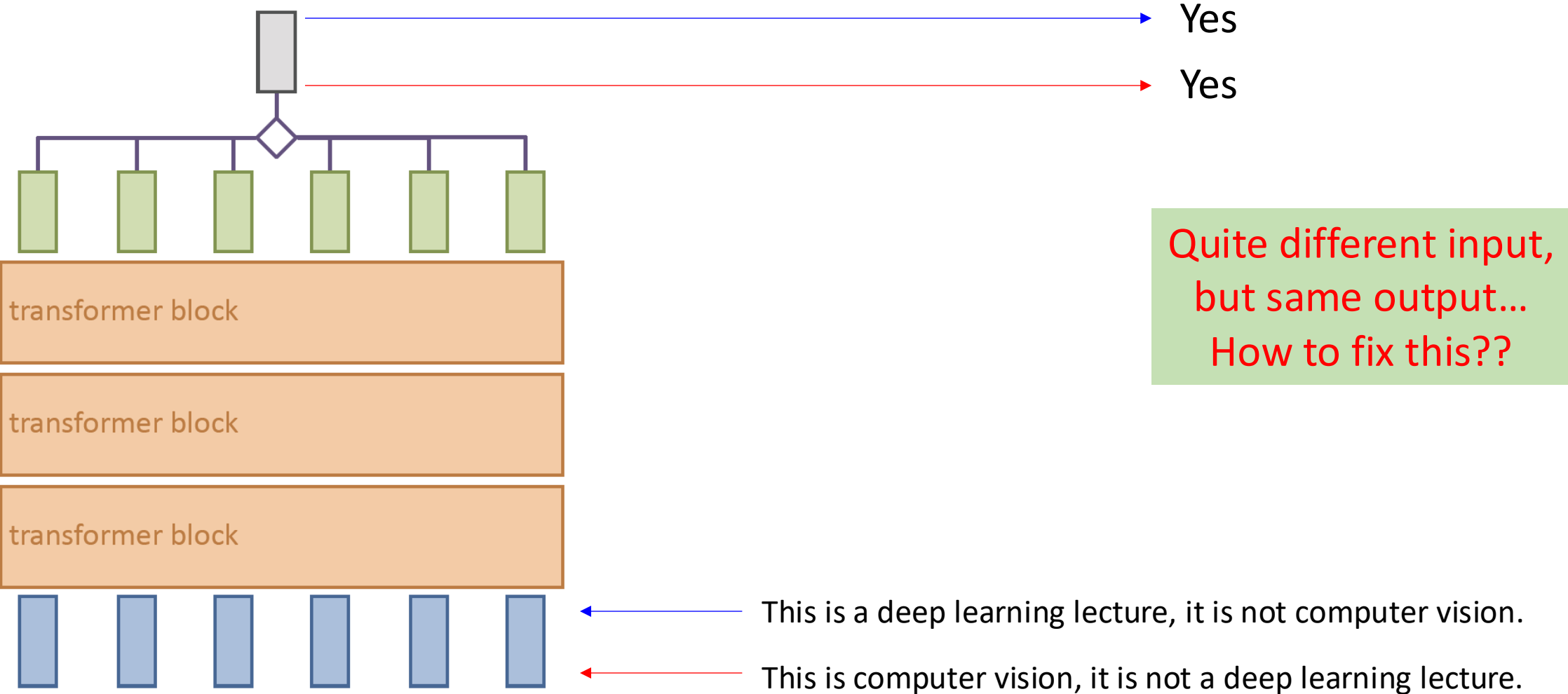
MASKING: MAKING SELF-ATTENTION CAUSAL



Self-attention - Equivariance

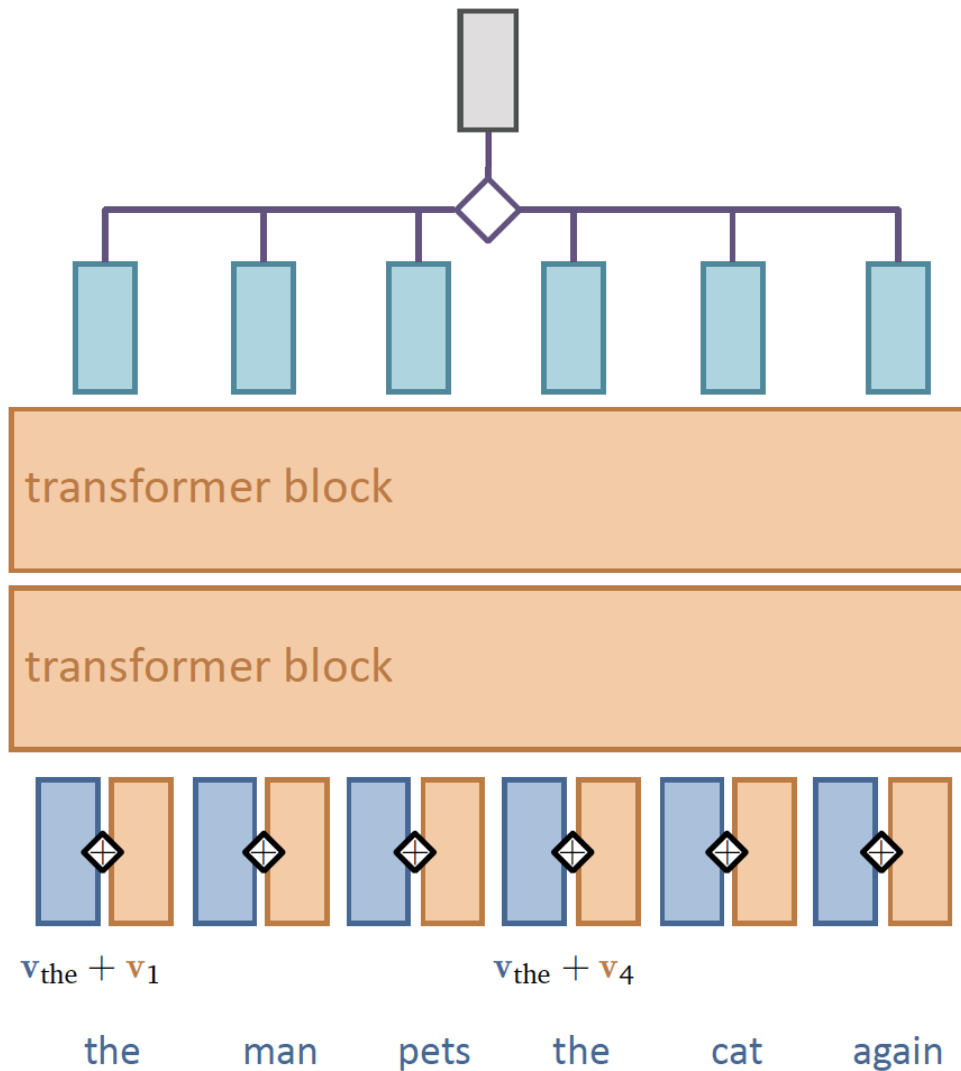


Self-attention - Equivariance



Breaking Equivariance

Position and order of words are the essential parts of any language



word embeddings:

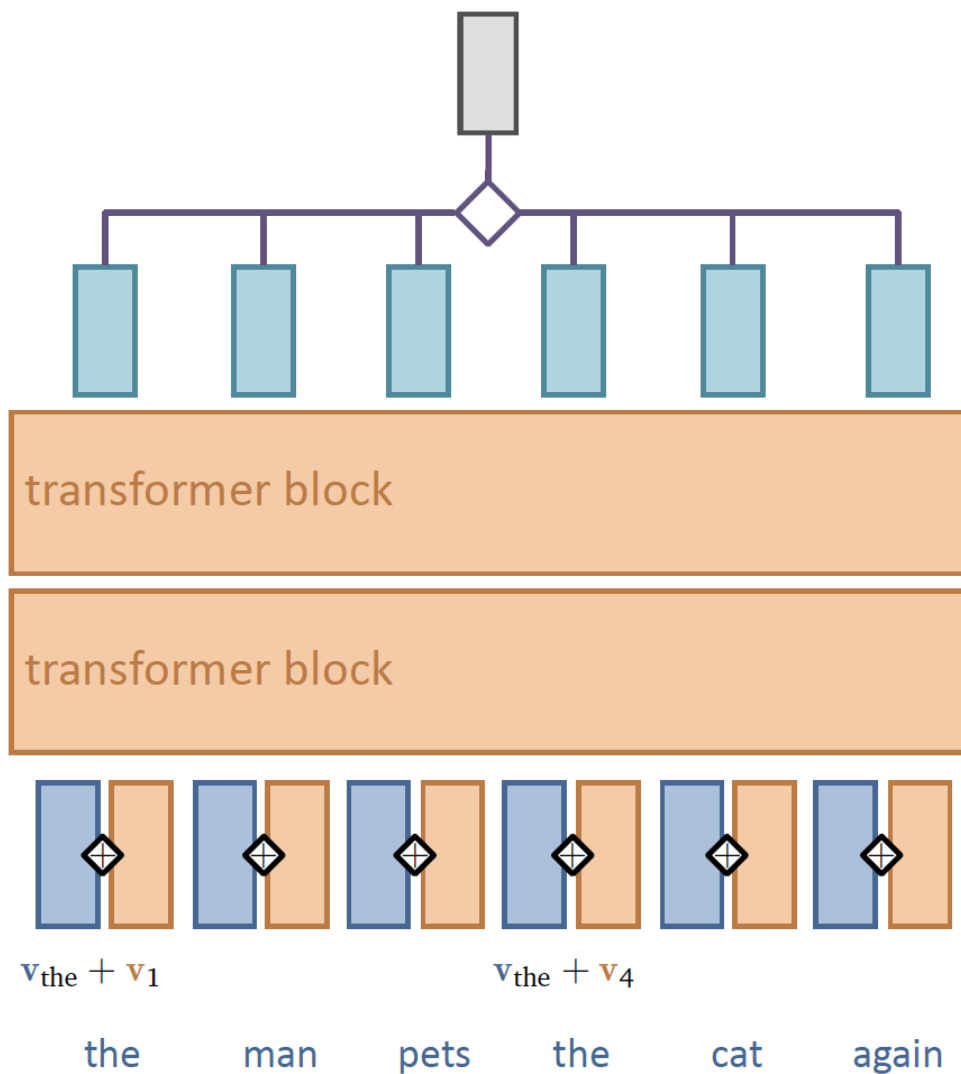
$v_{\text{the}}, v_{\text{man}}, v_{\text{pets}}, v_{\text{cat}}, v_{\text{again}}$

+

position embeddings:

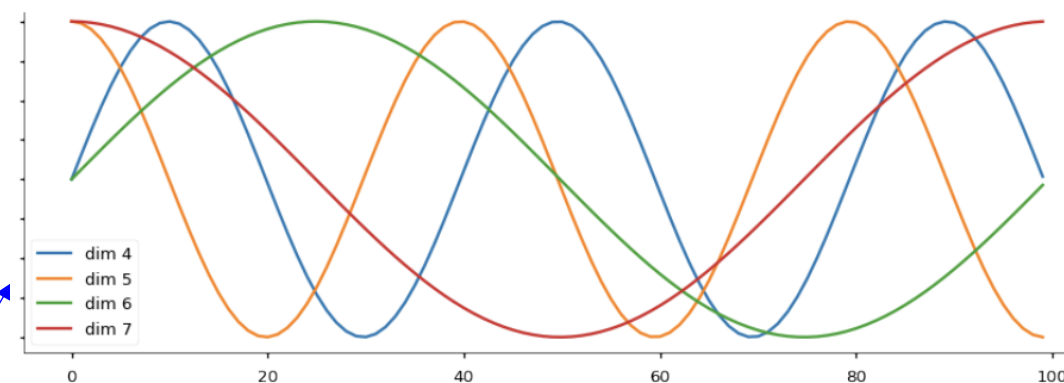
$v_1, v_2, v_3, v_4, v_5, \dots$

Breaking Equivariance

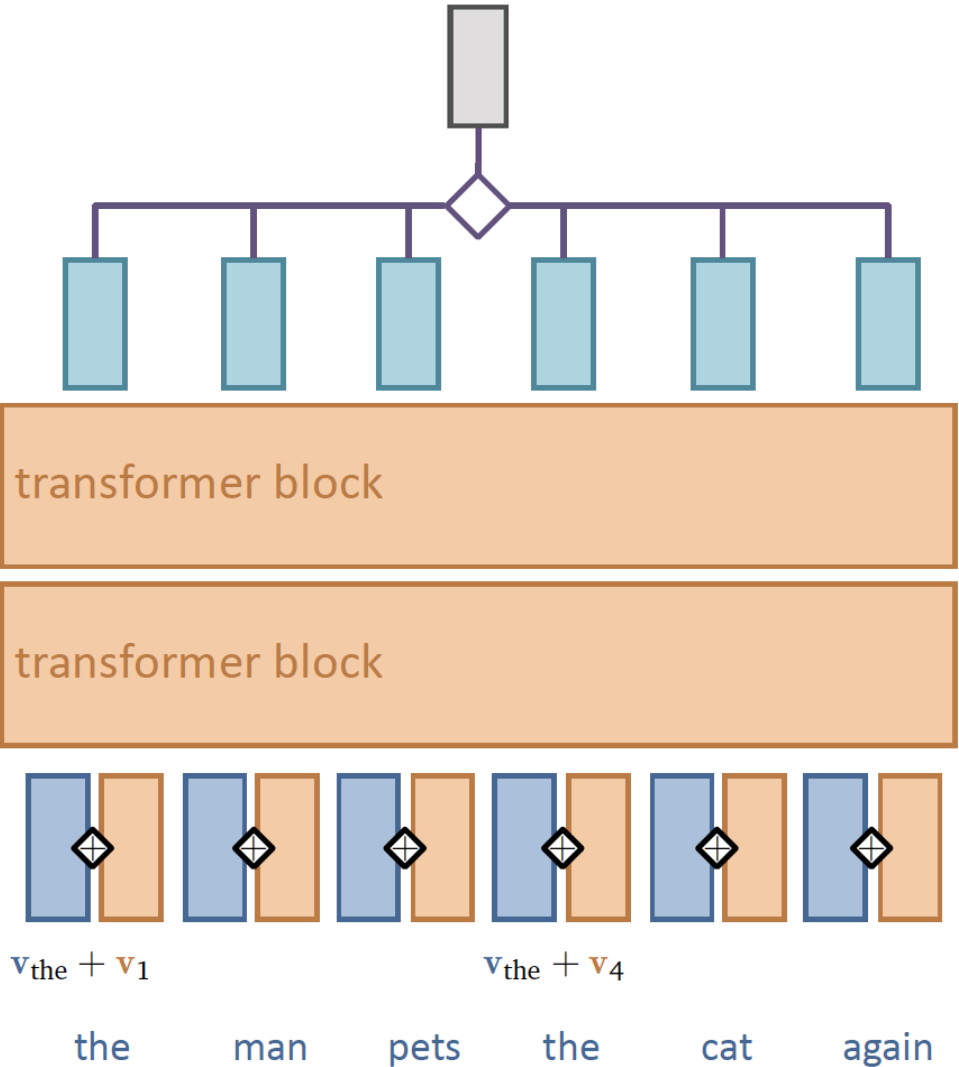


position encodings:

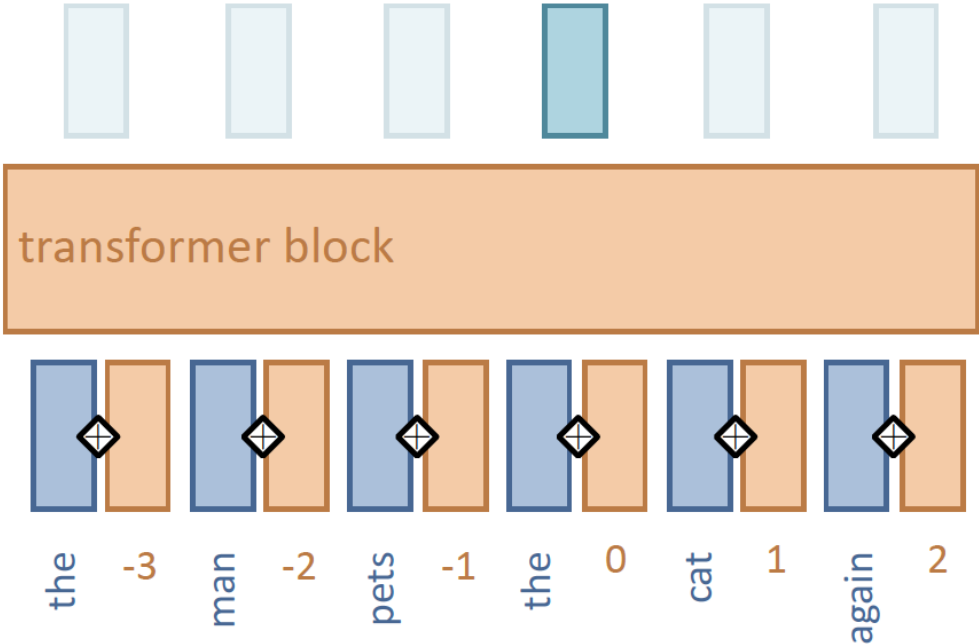
$v_1, v_2, v_3, v_4, v_5, \dots$



Breaking Equivariance



relative position:

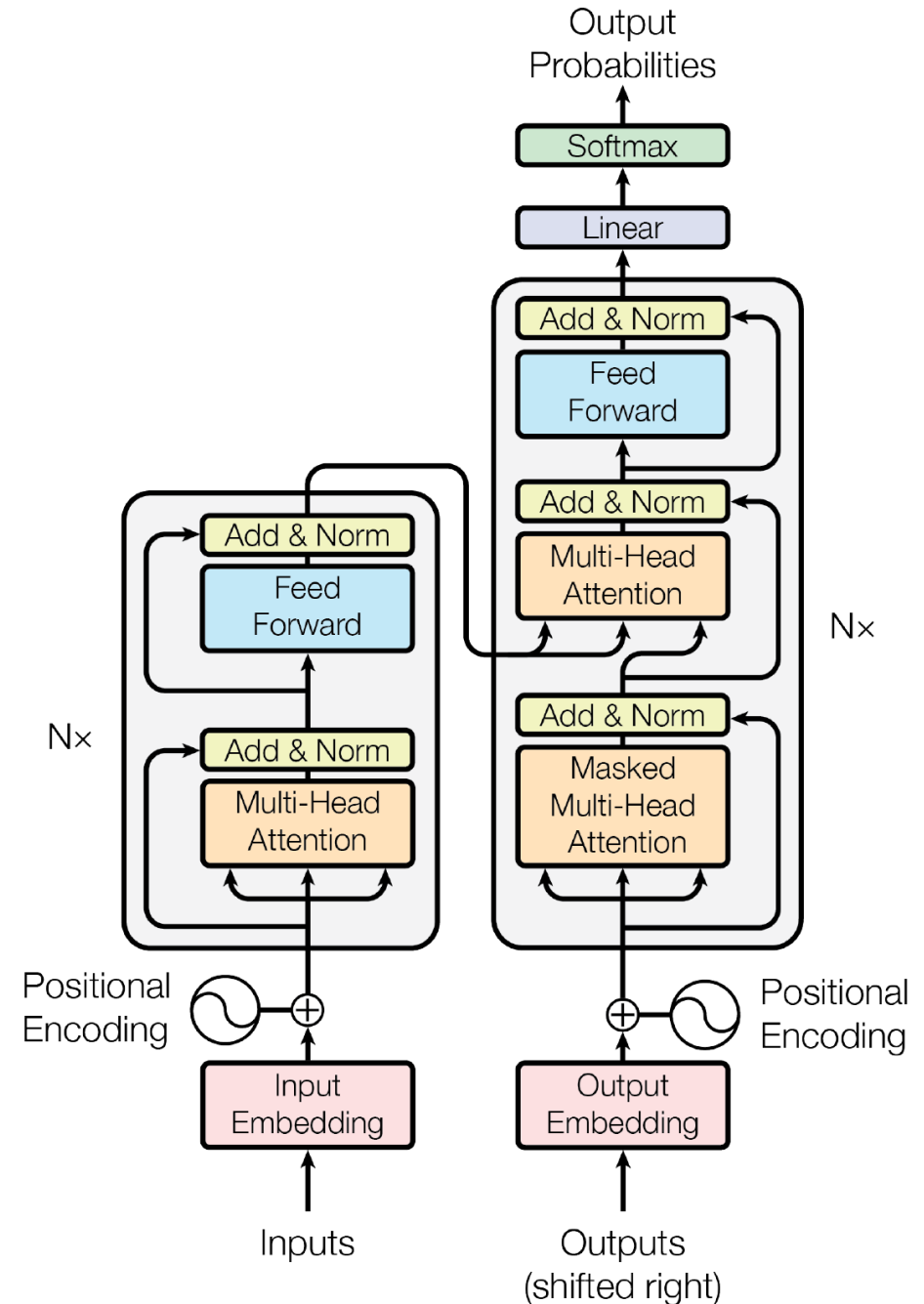


Transformers – Full Architecture

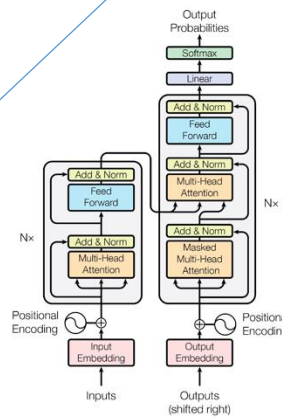
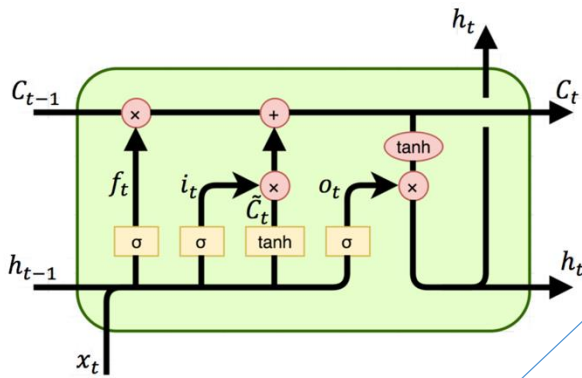
Attention is all you need (2017)

Parts we do not know yet:

- ~~○ Layer Normalisation~~
- ~~○ Multi-Head Attention~~
- ~~○ Masked Multi-Head Attention~~
- ~~○ Positional Encoding~~



RNN vs Transformers



Advantages of Transformers

- able to understand the relationship between sequential elements
- way more accurate in many cases
- can process and train more data in lesser time
- Support parallel processing

The Transformer architecture eliminates the time-dependent aspect of the RNN architecture by handling these aspects of learning in a completely separate architecture

Transformers are not better than traditional RNNs in all applications. RNNs still win in some contexts, but in those applications where they match or beat traditional RNNs they do so **with lower computational cost.**

Further Information - Codes

Self-attention implementation:

https://colab.research.google.com/github/mrm8488/shared_colab_notebooks/blob/master/basic_self_attention.ipynb

Cross-attention implementation:

<https://vaclavkosar.com/ml/cross-attention-in-transformer-architecture#:~:text=Cross%20attention%20is%3A,e.g.%20text%2C%20image%2C%20sound>

Bidirectional cross-attention implementation:

<https://github.com/lucidrains/bidirectional-cross-attention>

Transformers implementation:

<https://github.com/huggingface/transformers>

<https://paperswithcode.com/method/transformer>

<https://www.tensorflow.org/text/tutorials/transformer>

<https://kikaben.com/transformers-coding-details/>